Java Encapsulation: A Core OOP Principle

What is Encapsulation?

- Bundling data (variables) and methods (functions) that operate on that data into a single unit (a class).
- Restricts direct access to an object's internal components, preventing accidental data modification.

Purpose of Encapsulation

- **Data Hiding:** Conceals internal object representation; only necessary details are exposed via a public interface.
- **Increased Flexibility:** Allows internal implementation changes without affecting external code.
- **Improved Maintainability:** Simplifies code maintenance by keeping fields private and providing public getter/setter methods.

Implementing Encapsulation in Java

1. Declare class variables as `private`.
2. Provide `public` getter and setter methods to access and update the value of a private variable.

Example 1: Basic Encapsulation (`Student` Class)

```
public class Student {
    private String name;
    private int age;

    // Getter method for name
    public String getName() {
        return name;
    }

    // Setter method for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter method for age
    public int getAge() {
        return age;
    }

    // Setter method for age (with basic validation)
    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
```

```
    }
}
```

Example 2: Encapsulation with Validation (`BankAccount` Class)

```
public class BankAccount {
    private double balance;

    // Getter method for balance
    public double getBalance() {
        return balance;
    }

    // Setter method for balance with validation
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
        }
    }
}
```

Example 3: Encapsulation (`Account` Class)

```
// Java Program which demonstrate Encapsulation
class Account {

    // Private data members (encapsulated)
    private long accNo; // Account number
    private String name;
    private String email;
    private float amount;

    // Public getter and setter methods (controlled access)
    public long getAccNo() { return accNo; }
    public void setAccNo(long accNo) { this.accNo = accNo; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public float getAmount() { return amount; }
    public void setAmount(float amount) { this.amount = amount; }
}

// Main Class
public class Geeks
```

```
{
    public static void main(String[] args)
    {
        // Create an Account object
        Account acc = new Account();

        // Set values using setter methods (controlled access)
        acc.setAccNo(90482098491L);
        acc.setName("ABC");
        acc.setEmail("abc@gmail.com");
        acc.setAmount(100000f);

        // Get values using getter methods
        System.out.println("Account Number: " + acc.getAccNo());
        System.out.println("Name: " + acc.getName());
        System.out.println("Email: " + acc.getEmail());
        System.out.println("Amount: " + acc.getAmount());
    }
```

Tips and Best Practices

- **Use Private Fields:** Always declare class fields as `private` to protect them from external modification.
- **Provide Access Methods:** Use `public` getter and setter methods to control access.
- **Validation Logic:** Implement validation in setter methods to ensure consistent object state.
- **Immutable Classes:** Consider making classes immutable (without setters) if the object state should not change after creation, enhancing security.